Functions

- A function is a reusable block of JavaScript code designed to perform a particular task.
- Instead of writing the same statements repeatedly, we can place them inside a function and call the function whenever required.
- A function executes when it is called or invoked.

Why Functions Are Used

- To avoid repeating the same code.
- To divide a large program into smaller tasks.
- To improve code readability.
- To make code easier to test and maintain.
- To reuse the same logic with different values.

Function Syntax

```
function functionName(parameters) {
   // statements
}
```

function is the keyword used to declare a function. functionName is the name given to the function. parameters are optional inputs received by the function.

Function Declaration

A function declaration defines a function using the function keyword.

```
function greet() {
   console.log("Hello Students");
}
```

The above code only defines the function. The statements inside it execute when the function is called.

Calling a Function

Calling a function means executing the function.

```
function greet() {
   console.log("Hello Students");
}

greet();
```

Output: Hello Students

Type of Functions

1. Function Without Parameters

A function can perform a task without receiving any input values.

```
function welcome() {
   console.log("Welcome to JavaScript");
}

welcome();
```

2. Function With Parameters

Parameters are variables written in the function definition. They receive values when the function is called.

```
function greet(name) {
   console.log("Hello " + name);
}

greet("ABC");
```

Here, name is the parameter and "ABC" is the argument passed to the function.

Parameters and Arguments

Term	Meaning	Example
Parameter	Variable defined in the function declaration.	name in function greet(name)
Argument	Actual value passed when calling the function.	"ABC" in greet("ABC")

Function With Multiple Parameters

```
function add(a, b) {
   console.log(a + b);
}


add(10, 20);
```

Output: 30

Return Statement

The return statement sends a value from the function back to the place where the function was called.

```
function add(a, b) {
   return a + b;
}


let result = add(10, 20);
console.log(result);
```

Output: 30

- return ends the execution of the function.
- The returned value can be stored in a variable or used in another expression.
- A function does not need to use return if it only performs an action.

Function With Return Value

```
function square(number) {
   return number * number;
}

let result = square(5);
console.log(result);
```

Output: 25

Function Without Return Value

A function can perform an action without returning a value.

```
function displayMessage() {
   console.log("Welcome");
}

displayMessage();
```

Passing Different Values

The same function can be reused with different arguments.

```
function greet(name) {
   console.log("Hello " + name);
}

greet("ABC");
greet("DEF");
greet("GHI");
```

The function is written once but can be called multiple times with different values.

Local Variables in a Function

A variable declared inside a function is normally available only within that function.

```
function showAge() {
   let age = 20;
   console.log(age);
}

showAge();
```

- The variable age is local to showAge().
- It should not be directly accessed outside the function.

Default Parameters

A default parameter provides a value when an argument is not supplied. This feature is commonly used with modern JavaScript and is also covered under ES6 Features.

```
function greet(name = "Guest") {
   console.log("Hello " + name);
}

greet();
greet("ABC");
```

Output:
Hello Guest
Hello ABC

Function Expression / Lambda Function

A function can also be stored in a variable. This is called a function expression.

```
const add = function(a, b) {
   return a + b;
};

console.log(add(10, 20));
```

Output: 30

- The function is assigned to the variable add.
- The function can be called using add().

JS Objects

- JavaScript provides several built-in ways to store, organize, and work with data.
- In this topic, we learn the commonly used JavaScript objects and object-like structures: Object, Array, String, Date, Math, Map, and Set.

1. Object

An object is a collection of related data stored as key-value pairs. Each key is called a property, and the value can contain data such as a string, number, Boolean, array, or another object.

Creating an Object

```
let student = {
   name: "ABC",
   age: 20,
   course: "JavaScript"
};
```

Accessing Object Properties

Object properties can be accessed using dot notation or bracket notation.

```
console.log(student.name);
console.log(student["age"]);
```

Output:
ABC
20

Adding and Updating Properties

```
student.city = "Vijayawada";
student.age = 21;
```

```
console.log(student.city);
console.log(student.age);
```

- A new property can be added using the object name and property name.
- An existing property can be updated by assigning a new value.

Deleting an Object Property

```
delete student.city;
```

The delete operator removes a property from an object.

2. Array

An array is an ordered collection of values. It is used when multiple values need to be stored under one variable.

```
let marks = [80, 75, 90, 85];
```

Accessing Array Elements

Array indexing starts from 0.

```
console.log(marks[0]);
console.log(marks[2]);
```

Output:
80
90

Common Array Properties and Methods

Property / Method	Purpose	Example
length	Returns the number of elements.	marks.length
push()	Adds an element at the end.	marks.push(95)
pop()	Removes the last element.	marks.pop()
shift()	Removes the first element.	marks.shift()
unshift()	Adds an element at the beginning.	marks.unshift(70)

3. String

A string represents a sequence of characters or text. JavaScript provides several properties and methods for working with strings.

let message = "Hello JavaScript";

Accessing String Characters

console.log(message[0]);

Output: H

Common String Properties and Methods

Property / Method	Purpose	Example
length	Returns the number of characters.	message.length
toUpperCase()	Converts text to uppercase.	message.toUpperCase()
toLowerCase()	Converts text to lowercase.	message.toLowerCase()
charAt()	Returns the character at a position.	message.charAt(0)
includes()	Checks whether text is present.	message.includes("JavaScript")

4. Date

The Date object is used to work with dates and times in JavaScript.

let currentDate = new Date();
console.log(currentDate);

new Date() creates a Date object representing the current date and time.

Creating a Specific Date

```
let date = new Date("2026-10-03");
console.log(date);
```

Common Date Methods

Method	Purpose	Example
getFullYear()	Returns the year.	date.getFullYear()
getMonth()	Returns the month index.	date.getMonth()
getDate()	Returns the day of the month.	date.getDate()
getDay()	Returns the day of the week index.	date.getDay()
getHours()	Returns the hour.	date.getHours()
getMinutes()	Returns the minutes.	date.getMinutes()

Note: getMonth() returns a zero-based month index, where January is 0 and December is 11.

5. Math

Math is a built-in JavaScript object that provides mathematical constants and methods. It is used for common mathematical calculations.

```
console.log(Math.PI);
console.log(Math.sqrt(25));
console.log(Math.pow(2, 3));
```

Output:

```
3.141592653589793
5
8
```

Common Math Properties and Methods

Property / Method	Purpose	Example
Math.PI	Returns the value of π.	Math.PI

Math.sqrt()	Returns the square root.	Math.sqrt(25)
Math.pow()	Returns a number raised to a power.	Math.pow(2, 3)
Math.round()	Rounds to the nearest integer.	Math.round(4.6)
Math.floor()	Rounds down.	Math.floor(4.9)
Math.ceil()	Rounds up.	Math.ceil(4.1)
Math.max()	Returns the largest value.	Math.max(10, 20, 30)
Math.min()	Returns the smallest value.	Math.min(10, 20, 30)
Math.random()	Returns a random number from 0 up to, but not including, 1.	Math.random()

6. Map

A Map is a collection of key-value pairs. Unlike a regular object, a Map can use different types of values as keys.

```
let student = new Map();

student.set("name", "ABC");
student.set("age", 20);

console.log(student.get("name"));
console.log(student.get("age"));
```

Output:
ABC
20

Common Map Methods

Method	Purpose	Example
set()	Adds or updates a key-value pair.	student.set("age", 21)
get()	Returns the value for a key.	student.get("age")
has()	Checks whether a key exists.	student.has("name")

delete()	Removes a key-value pair.	student.delete("age")
clear()	Removes all key-value pairs.	student.clear()
size	Returns the number of entries.	student.size

7. Set

A Set is a collection of unique values. Duplicate values are automatically ignored.

```
let numbers = new Set();

numbers.add(10);
numbers.add(20);
numbers.add(10);

console.log(numbers);
```

The value 10 is stored only once because a Set does not allow duplicate values.

Common Set Methods

Method	Purpose	Example
add()	Adds a value.	numbers.add(30)
has()	Checks whether a value exists.	numbers.has(20)
delete()	Removes a value.	numbers.delete(20)
clear()	Removes all values.	numbers.clear()
size	Returns the number of unique values.	numbers.size

Browser Object Model (BOM)

- The Browser Object Model (BOM) allows JavaScript to interact with the web browser. It provides objects that represent the browser window and browser-related information.

- BOM is used to working with the browser window.
- It provides information about the current URL, browser, screen, and browsing history.
- The window object is the main object of the BOM.

BOM Structure

window
- location
- history
- navigator
- screen

The window object represents the browser window. Other browser-related objects can be accessed through window.

1. Window Object

The window object represents the browser window in which the web page is displayed. It is the top-level object available in browser-side JavaScript.

```
window.alert("Welcome");
```

In many cases, window can be omitted.

```
alert("Welcome");
```

Common Window Properties

Property	Purpose
innerWidth	Returns the width of the browser's content area.
innerHeight	Returns the height of the browser's content area.

```
console.log(window.innerWidth);
console.log(window.innerHeight);
```

Common Window Methods

Method	Purpose
alert()	Displays a message box.
confirm()	Displays OK and Cancel buttons.
prompt()	Accepts input from the user.
open()	Opens a new browser window or tab.
close()	Attempts to close a window opened by script.

alert()

The alert() method displays a message to the user.

```
alert("Welcome to JavaScript");
```

confirm()

The confirm() method displays a dialog box with OK and Cancel buttons. It returns true when OK is selected and false when Cancel is selected.

```
let result = confirm("Do you want to continue?");
console.log(result);
```

prompt()

The prompt() method displays a dialog box that allows the user to enter a value.

```
let name = prompt("Enter your name");
console.log(name);
```

2. Location Object

The location object contains information about the current page URL and can be used to navigate to another page.

console.log(window.location.href);

Common Location Properties

Property	Purpose
href	Returns the complete URL.
hostname	Returns the domain name.
pathname	Returns the path of the current page.
protocol	Returns the URL protocol.

Common Location Methods

Method	Purpose
assign()	Loads another page.
reload()	Reloads the current page.
replace()	Loads another page and replaces the current history entry.

location.assign("https://example.com");
location.reload();

3. History Object

It represents the pages visited in the current browser session. It can be used to move backward or forward through the browsing history.

Common History Methods

Method	Purpose
back()	Moves to the previous page.
forward()	Moves to the next page.
go()	Moves to a particular position in the session history.

history.back();
history.forward();
history.go(-1);

history.go(-1) moves one page backward in the session history.

4. Navigator Object

The navigator object provides information about the browser and its environment.

Common Navigator Properties

Property	Purpose
userAgent	Returns information about the browser's user-agent string.
language	Returns the browser's preferred language.
onLine	Indicates whether the browser reports that it is online.

```
console.log(navigator.userAgent);
console.log(navigator.language);
console.log(navigator.onLine);
```

Note: Browser identification should not normally rely only on userAgent because its value can vary or be modified.

5. Screen Object

The screen object provides information about the user's screen.

Common Screen Properties

Property	Purpose
width	Returns the screen width in CSS pixels.
height	Returns the screen height in CSS pixels.
availWidth	Returns the available screen width.
availHeight	Returns the available screen height.
colorDepth	Returns the color depth of the screen.

```
console.log(screen.width);
console.log(screen.height);
```

Document Object Model (DOM)

- The Document Object Model (DOM) represents an HTML page as a document that JavaScript can access and modify.

- JavaScript can use the DOM to read HTML content.
- JavaScript can change the content of HTML elements.
- JavaScript can create dynamic changes in a web page.
- The document object is used to work with the HTML document.

DOM Methods and Properties

The following methods and properties are important for basic DOM programming.

Methods

1. document.write()

The document.write() method writes content directly into the HTML document.

```
document.write("Hello Students");
```

Output: Hello Students

2. document.writeln()

The document.writeln() method writes text to the document and adds a newline character after the text.

```
document.writeln("Hello");
document.writeln("Students");
```

The method is similar to document.write(), but it writes a newline after the supplied text.

3. document.getElementById()

The getElementById() method finds an HTML element using its id attribute.

```
<p id="message">Hello</p>

<script>
   let element = document.getElementById("message");
   console.log(element);
</script>
```

The method returns the element whose id matches the supplied value.

Changing an Element Using getElementById()

```
<p id="message">Old Message</p>

<script>
   let element = document.getElementById("message");
   element.innerHTML = "New Message";
</script>
```

The content of the paragraph changes from Old Message to New Message.

4. document.getElementsByClassName()

The getElementsByClassName() method finds HTML elements that have a specified class name.

```
<p class="item">One</p>
<p class="item">Two</p>
<script>
   let elements = document.getElementsByClassName("item");
   console.log(elements);
</script>
```

The method returns to a collection of elements that have the specified class.

5. document.getElementsByTagName()

The getElementsByTagName() method finds HTML elements using their tag name, such as p, h1, div, or li.

```
<p>First Paragraph</p>
<p>Second Paragraph</p>

<script>
   let paragraphs = document.getElementsByTagName("p");
   console.log(paragraphs);
</script>
```

The method returns to a collection containing all matching elements.

Note

Name	Main Purpose
document.write()	Writes content to the document.
document.writeln()	Writes content with a newline.
document.getElementById()	Finds an element using its id.
document.getElementsByClassName()	Finds elements using a class name.
document.getElementsByTagName()	Finds elements using a tag name.

Properties

Properties are used to read or change information associated with an HTML element.

1. innerHTML

The innerHTML property gets or changes the HTML content inside an element.

```
<p id="message">Hello</p>

<script>
   document.getElementById("message").innerHTML = "<b>Welcome</b>";
</script>
```

The content inside the paragraph becomes bold because HTML markup is interpreted.

Reading innerHTML

```
let content = document.getElementById("message").innerHTML;
console.log(content);
```

innerHTML can read the HTML markup contained inside an element.

2. innerText

The innerText property gets or changes the visible text inside an element.

```
<p id="message">Hello Students</p>

<script>
   document.getElementById("message").innerText = "Welcome Students";
</script>
```

The visible text changes to Welcome Students.

innerHTML vs innerText

Property	Purpose	Example
innerHTML	Gets or sets HTML content inside an element.	element.innerHTML = "Hello";
innerText	Gets or sets visible text inside an element.	element.innerText = "Hello";

Form Validation

- Form validation is the process of checking the data entered by a user before the form is submitted.

- Validation helps identify incorrect or incomplete input.
- It can provide a message to the user when the entered data is invalid.
- Validation can be performed before sending form data to the server.

Basic Form Validations

- Name
- Password
- Mobile
- Email
- Date

1. Name Validation

- Name validation checks whether the user has entered a name and whether the name contains acceptable characters.

- Name should not be empty and can be checked using a regular expression that allows alphabetic characters and spaces.

```
let name = document.getElementById("name").value;

let namePattern = /^[A-Za-z ]+$/;

if (name === "") {
   console.log("Name is required");
} else if (!namePattern.test(name)) {
   console.log("Enter a valid name");
} else {
   console.log("Valid name");
}
```

Name Validation Rules

- Name should not be empty.
- Name should contain alphabetic characters and spaces.
- Special characters and numbers can be rejected when the application requires a simple alphabetic name.

2. Password Validation

- Password validation checks whether the user has entered a password and whether it satisfies the required password rules.

- A common basic rule is to require a minimum number of characters. Applications can define stronger rules depending on their security requirements.

```
let password = document.getElementById("password").value;


if (password === "") {
   console.log("Password is required");
} else if (password.length < 8) {
   console.log("Password must contain at least 8 characters");
} else {
   console.log("Valid password");
}
```

Password Validation Rules

- Password should not be empty.
- A minimum length should be defined by the application.
- For stronger validation, applications may require uppercase letters, lowercase letters, numbers, and special characters.
- Passwords should not be displayed openly on the page.

3. Mobile Validation

- Mobile validation checks whether the entered mobile number follows the format expected by the application.

- For a simple example, a 10-digit numeric mobile number can be checked using a regular expression.

```
let mobile = document.getElementById("mobile").value;

let mobilePattern = /^[0-9]{10}$/;

if (mobile === "") {
   console.log("Mobile number is required");
} else if (!mobilePattern.test(mobile)) {
   console.log("Enter a valid 10-digit mobile number");
} else {
   console.log("Valid mobile number");
}
```

Mobile Validation Rules

- Mobile number should not be empty.
- For this basic example, exactly 10 digits are required.
- Only numeric digits are accepted.
- The exact mobile-number format should match the application's requirements and country rules.

4. Email Validation

- Email validation checks whether the entered value follows a basic email address pattern.

```
let email = document.getElementById("email").value;

let emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
```

```
if (email === "") {
   console.log("Email is required");
} else if (!emailPattern.test(email)) {
   console.log("Enter a valid email address");
} else {
   console.log("Valid email");
}
```

Email Validation Rules

- Email should not be empty.
- A basic email pattern should be checked.
- The value should contain an appropriate local part, @ symbol, and domain part.
- Client-side pattern checking is only a basic format check; it does not prove that an email address exists.

5. Date Validation

- Date of Birth validation checks whether a date has been entered and whether it satisfies the application's rules.

- A simple validation can check that the field is not empty. A common additional rule is that the Date of Birth should not be a future date.

```
let dob = document.getElementById("dob").value;
let today = new Date();
let selectedDate = new Date(dob);


if (dob === "") {
   console.log("Date of Birth is required");
} else if (selectedDate > today) {
   console.log("Date of Birth cannot be a future date");
} else {
```

```
    console.log("Valid Date of Birth");
}
```

Date Validation Rules

- Date of Birth should not be empty.
- The selected date should be a valid date.
- The Date of Birth should not be in the future.
- An application may also define an age range when required.

Note:

Regular Expression (Regex)

A regular expression is a pattern used to match and validate text. In form validation, regular expressions are commonly used for fields such as name, mobile number, and email.

```
let pattern = /^[0-9]{10}$/;


console.log(pattern.test("9876543210"));
```

The test() method checks whether the supplied value matches the pattern.

ES6 Features

- ES6 stands for ECMAScript 6 and is also known as ECMAScript 2015. It is a major version of JavaScript that introduced features designed to make JavaScript code shorter, cleaner, and easier to maintain.

ES6 Features

Feature	Main Idea
let and const	Modern variable declarations
Arrow Functions	Short function syntax
Template Literals	Strings with variables and expressions
Destructuring	Extract values from arrays and objects
Default Parameters	Default function arguments
Rest and Spread	Collect and expand values
Classes	Blueprint for objects
Modules	Share and reuse code
for...of	Iterate through values
Map	Key-value collection
Set	Unique-value collection

1. let and const

ES6 introduced let and const for declaring variables. Both are block-scoped.

```
let age = 25;
age = 30;
const name = "ABC";
console.log(age);
console.log(name);
```

Output:
30 ABC

- let → the value can be reassigned.
- const → the variable cannot be reassigned.
- Both are block-scoped.

- They are generally preferred over var in modern JavaScript.

Remember: let → can change | const → cannot be reassigned

2. Arrow Functions

Arrow functions provide a shorter syntax for writing functions.

```
const add = (a, b) => a + b;

console.log(add(10, 20));
```

Output: 30

- Uses the => syntax.
- Makes function expressions shorter.
- Commonly used with arrays and callbacks.
- Arrow functions have lexical this behavior.

Remember: Arrow Function → Shorter way to write a function

3. Template Literals

Template literals make it easy to create strings containing variables and expressions.

```
let name = "ABC";
let age = 20;

console.log(`Name: ${name}, Age: ${age}`);
```

Output: Name: ABC, Age: 20

- Uses backticks (` `).
- Use ${} to insert variables or expressions.
- Reduces string concatenation using +.

Remember: Template Literal → String + variables/expressions using backticks

4. Destructuring

Destructuring allows values to be extracted from arrays or objects into variables.

```
let numbers = [10, 20];
let [a, b] = numbers;

let student = { name: "ABC", age: 20 };
let { name, age } = student;

console.log(a, b);
console.log(name, age);
```

Output:
10 20
ABC 20

- Array destructuring uses [].
- Object destructuring uses {}.
- Makes extracting values shorter and cleaner.

Remember: Destructuring → Extract values from arrays/objects easily

5. Default Parameters

Default parameters provide a default value when a function argument is not supplied.

```
function greet(name = "Guest") {
   console.log(`Hello ${name}`);
}

greet();
greet("ABC");
```

Output:
Hello Guest
Hello ABC

- The default value is used when the argument is missing or undefined.
- Makes functions easier and safer to use.

Remember: Default Parameter → Default value for a missing argument

6. Rest and Spread Operator

The ... syntax is used as Rest to collect values and Spread to expand values.

Rest Operator

```
function sum(...numbers) {
   return numbers[0] + numbers[1] + numbers[2];
}


console.log(sum(10, 20, 30));
```

Output: 60

Spread Operator

```
let nums = [10, 20, 30];
let newNums = [...nums, 40];


console.log(newNums);
```

Output: [10, 20, 30, 40]

- Rest → collects multiple values into one variable.
- Spread → expands elements of an array or object.
- Both use the ... syntax.

Remember: Rest → Collect | Spread → Expand

7. Classes

A class provides a blueprint for creating objects. ES6 introduced class syntax to make object-oriented programming more convenient.

```
class Student {
   constructor(name, age) {
      this.name = name;
      this.age = age;
   }

   display() {
      console.log(this.name, this.age);
   }
}

let student = new Student("ABC", 20);
student.display();
```

Output: ABC 20

- class defines the class.
- constructor() initializes object data.
- this refers to the current object.
- new creates an object from the class.

8. Modules

Modules allow JavaScript code to be divided into separate files and shared between them using export and import.

Export

```
export const name = "ABC";
```

Import

```
import { name } from "./student.js";

console.log(name);
```

- export makes code available to other modules.
- import uses code exported from another module.
- Modules help organize large applications into separate files.

9. for...of

The for...of loop is used to iterate over the values of an iterable such as an array.

```
let numbers = [10, 20, 30];

for (let number of numbers) {
   console.log(number);
}
```

Output:
10 20 30

- for...of gives values from the iterable.
- It is commonly used with arrays and other iterable collections.

10. Map

Map is a collection that stores data as key-value pairs.

```
let student = new Map();

student.set("name", "ABC");
student.set("age", 20);

console.log(student.get("name"));
```

Output: ABC

- set() adds or updates a key-value pair.
- get() retrieves a value using its key.
- Map can use different types of values as keys.

Remember: Map → Key-value pairs

11. Set

Set is a collection that stores unique values.

```
let numbers = new Set();


numbers.add(10);
numbers.add(20);
numbers.add(10);


console.log(numbers);
```

The duplicate value 10 is stored only once.

- add() adds a value.
- has() checks whether a value exists.
- Set does not store duplicate values.

Remember: Set → Unique values

MODULE 2
GIT

List of Topics

1. Introduction to VCS and GIT
2. GIT Installation
3. GITHUB Account Creation
4. GIT Configuration
5. GIT Basics
6. Working with Branches
7. Remote Repository
8. GIT Collaboration
9. Undo Changes
10. GIT Tools

Introduction to Version Control System (VCS)

- A Version Control System (VCS) is a software tool used to track changes made to files and source code over time.
- It helps developers maintain different versions of a project and understand what changes were made.
- VCS is especially useful when multiple developers work on the same project because it helps them manage changes, collaborate, and recover earlier versions when required.

Why Version Control is Needed

Without version control, developers may create many copies of the same project, such as:

- project_final
- project_final_new
- project_final_latest

This approach makes it difficult to identify which version is correct and who made a particular change.

A VCS provides a systematic way to:

- Track changes in source code.
- Maintain the history of a project.

- Identify who made a change.
- Restore an earlier version when required.
- Allow multiple developers to work on the same project.
- Create separate development paths using branches.

Types of Version Control Systems

Type	Description	Example
Local VCS	Version history is maintained on the local computer.	RCS
Centralized VCS	A central server stores the main version history.	SVN
Distributed VCS	Each developer can have a complete repository with its history.	GIT

What is GIT

- GIT is a distributed version control system used to track changes in source code and manage different versions of a project.
- GIT was originally created by Linus Torvalds in 2005
- GIT is commonly used with platforms such as GitHub, GitLab, and Bitbucket for remote repository hosting and collaboration. (GitHub is preferable)

Why is GIT Used

- To maintain the history of source-code changes.
- To work on different features separately using branches.
- To compare changes between versions.
- To restore previous versions of files.
- To support collaboration between developers.
- To work locally and synchronize changes with remote repositories.

Features of GIT

- Distributed – Each developer can have a complete local repository.
- Version History – Changes can be recorded and reviewed.
- Branching – Developers can create separate branches for features or fixes.
- Merging – Changes from different branches can be combined.
- Collaboration – Multiple developers can work on the same project.
- Data Integrity – GIT uses checksums to identify and protect stored content.

GIT Installation

- Before using GIT commands, GIT must be installed on the computer. After installation, the GIT command can be used from a terminal or command prompt.

What is GIT Installation

GIT Installation is the process of installing the GIT software on a computer so that GIT commands can be executed and repositories can be managed.

Installing GIT on Windows

The basic installation steps are:

1. Open a web browser.
2. Go to the official GIT website.
3. Download the GIT installer for Windows.
4. Run the downloaded installer.
5. Follow the installation wizard.
6. Keep the default options unless a specific configuration is required.
7. Complete the installation.

Verify GIT Installation

After installation, open Command Prompt, PowerShell, Git Bash, or another terminal and execute:

```
git --version
```

If GIT is installed correctly, the command displays the installed GIT version.

```
git version 2.x.x
```

The exact version number depends on the version installed on the computer.

GitHub Account Creation

- GitHub is a platform used to host GIT repositories and support software-development collaboration. A GitHub account allows a user to create and manage repositories on GitHub.

GIT and GitHub

GIT	GitHub
Version control software	Online platform for hosting and collaborating GIT repositories.

Runs on the developer's computer.	Provides repositories and collaboration features through an online service.
Tracks changes and creates commits.	Can store and share repositories remotely.

Steps to Create a GitHub Account

1. Open a web browser.
2. Go to the official GitHub website.
3. Select the option to create a new account.
4. Enter the required email address.
5. Create a password.
6. Choose a GitHub username.
7. Complete the required verification steps.
8. Verify the email address if requested.
9. Complete the account setup.

Choosing a GitHub Username

A username identifies the user's GitHub account and appears in the user's GitHub profile and repository URLs.

- Choose a professional and easy-to-remember username.
- Avoid unnecessarily complicated names.
- Remember that the username can become part of repository links.

GitHub Repository

A repository on GitHub is an online location where a project's files and GIT history can be stored.

For example, a student may create a repository named:

```
student-management-project
```

GIT Configuration

- After installing GIT, basic configuration should be performed before creating commits. The most important configuration is the user's name and email address.

Why GIT Configuration is Needed

- GIT records the author information along with commits. Configuring the user name and email helps identify who created a particular commit.

Check Existing Configuration

To display the current GIT configuration, use:

```
git config --list
```

To check the configured user name:

```
git config user.name
```

To check the configured email:

```
git config user.email
```

Configure User Name

Use the following command:

```
git config --global user.name "Your Name"
```

Configure Email

Use the following command:

```
git config --global user.email "you@example.com"
```

The --global option applies the configuration to the user's GIT configuration for repositories on that computer.

Example

```
git config --global user.name "abc"
git config --global user.email "abc@gmail.com"
```

Verify Configuration

After configuring the name and email, verify them using:

```
git config --global user.name
git config --global user.email
```

Local Repository Configuration

GIT also allows configuration to be applied only to a particular repository. This is useful when a repository needs settings different from the global configuration.

```
git config user.name "Project User"
```

```
git config user.email "project@example.com"
```

A repository-specific configuration takes precedence over the corresponding global setting for that repository.

Global vs Local Configuration

Configuration	Scope	Command Example
Global	Applies to repositories for the current user on the computer.	git config --global user.name "Your Name"
Local	Applies only to the current repository.	git config user.name "Project User"

Useful GIT Configuration Commands

Command	Purpose
git config --list	Displays configuration settings.
git config user.name	Displays the name configured for the current repository.
git config user.email	Displays the email configured for the current repository.
git config --global user.name "Name"	Sets the global user name.
git config --global user.email "Email"	Sets the global email.

GIT Basics

- GIT Basics covers the fundamental commands required to start working with a project using GIT. These commands help a developer create a repository, check file status, prepare changes, save changes as commits, and view the history of committed changes.

1. GIT Installation

Command

```
git --version
```

Purpose of the Command

- Check whether GIT is installed and displays the installed GIT version.

Example

```
git --version
```

Example output:
git version 2.x.x

2. GIT Configuration

Command

```
git config --global user.name "Your Name"
git config --global user.email "your@email.com"
```

Purpose of the Command

- Configures the name and email associated with commits.

Example

```
git config --global user.name "Rahul Kumar"
git config --global user.email "rahul@example.com"
```

3. GIT Initialization

Command

```
git init
```

Purpose of the Command

- Creates a new local GIT repository in the current project folder.

Example

```
cd StudentManagement
git init
```

4. GIT Status

Command

```
git status
```

Purpose of the Command

- Shows the current state of the working directory and staging area.

Example

```
git status
```

5. GIT Staging

Staging selects changes that should be included in the next commit. The staging area is between the working directory and the repository.

A. Adding a Single File

Command

```
git add Student.java
```

Purpose

Stages only Student.java.

B. Adding Multiple Files

Command

```
git add Student.java Course.java
```

Purpose

Stages the specified files.

C. Adding All Files

Command

```
git add .
```

Purpose

Stages all applicable changes in the current directory and its subdirectories.

6. GIT Commit

Command

```
git commit -m "Commit message"
```

Purpose of the Command

- Records the currently staged changes in the local repository.

Example

```
git commit -m "Added student class"
```

7. GIT Commit History

Detailed History

Command

```
git log
```

Purpose of the Command

- Displays the commit history of the current repository.

Example

```
git log
```

Short History

Command

```
git log --oneline
```

Purpose

Displays each commit in a compact single-line format.

GIT Basics – Command Summary

Command	Purpose
git --version	Checks installed GIT version.
git config	Configures GIT settings.
git init	Creates a local GIT repository.
git status	Shows repository/worktree state.
git add <file>	Stages one file.
git add file1 file2	Stages selected files.
git add .	Stages all applicable changes.
git commit -m	Records staged changes.
git log	Shows detailed history.
git log --oneline	Shows compact history.

Working with Branches

- GIT branch is a separate line of development in a repository. Branches allow developers to work on features, fixes, or experiments without directly changing the main development line.
- A branch can be created from an existing point in the project's history. Developers can work on the branch, create commits, and later merge the changes into another branch.

Why Branches are Used

- To develop new features separately.
- To fix bugs without disturbing the main development line.
- To allow multiple developers to work on different tasks.
- To test or experiment with changes safely.
- To organize project development.
- To combine completed work using merging.

Understanding the Main Branch

The main branch is commonly used as the primary branch of a repository. Depending on the repository or project, it may be named main or another name.

The important idea is that developers can create separate branches from the main development line and work independently.

```
main
  |
  +------ feature-login
  |
  +------ feature-payment
```

Checking the Current Branch

Command

```
git branch
```

Purpose of the Command

- The git branch command displays local branches. The current branch is normally indicated with an asterisk (*).

Example

```
git branch
```

```
* main
  feature-login
  feature-payment
```

In this example, the asterisk indicates that main is the current branch.

Creating a New Branch

Command

```
git branch feature-login
```

Purpose of the Command

- Creates a new branch named feature-login without automatically switching to it.

Example

```
git branch feature-login
git branch
```

After creation, the new branch appears in the branch list, but the current branch remains unchanged.

Switching to a Branch

Command

```
git switch feature-login
```

Purpose of the Command

- Switches the working directory to the specified existing branch.

Example

```
git switch feature-login
git branch
```

The asterisk now appears next to feature-login.

Creating and Switching to a Branch

Command

```
git switch -c feature-login
```

Purpose of the Command

- Creates a new branch and switches to it in one command.

Example

```
git switch -c feature-login
git branch
```

This is a convenient command when starting work on a new feature.

Note: Alternative Command

```
git checkout -b feature-login
```

This older command can also create and switch to a branch. For new classroom examples, git switch is clearer because it is specifically designed for branch switching.

Viewing Branches

Local Branches

```
git branch
```

Purpose: Displays local branches.

Branches with More Information

```
git branch -v
```

Purpose: Displays local branches with information about their latest commits.

Working with a Branch

Once a developer switches to a branch, changes can be made and committed normally.

```
git switch feature-login

# Modify project files

git status
git add .
git commit -m "Added login feature"
```

The commit is created on the current branch. The main branch is not automatically updated by this commit.

Note: Branch and Commit Relationship

Each branch points to a line of commits. When a new commit is created on a branch, that branch moves forward to include the new commit.

```
main
A --- B

feature-login
A --- B --- C --- D
```

Here, commits C and D belong to the feature-login development line. The main branch still points to B until the feature branch is merged.

Merging Branches

Merging is the process of combining changes from one branch into another branch.

Basic Workflow

```
git switch main
git merge feature-login
```

Purpose of git merge

The git merge command combines the changes from the specified branch into the currently switched branch.

Example

```
git switch main
git merge feature-login
```

Here, feature-login is merged into main. The developer must first switch to the branch that should receive the changes.

Note:

- The branch being merged into is the current branch.
- The branch named in git merge is the source of the changes.
- After a successful merge, the changes from the source branch become part of the target branch.

Renaming a Branch

Command

```
git branch -m old-name new-name
```

Purpose of the Command

Renames a local branch.

Example

```
git branch -m feature-login login-feature
```

Deleting a Branch

Command

```
git branch -d feature-login
```

Purpose of the Command

- Deletes the specified local branch when its changes have been merged or Git considers it safe to delete.

Example

```
git switch main
git branch -d feature-login
```

A branch should normally be deleted after its work has been merged and is no longer required.

Note: Force Deleting a Branch

```
git branch -D feature-login
```

Purpose: Force deletes the local branch. This should be used carefully because it can remove a branch that contains unmerged work.

Remote Repository

- A remote repository is a GIT repository stored on another system or on an online repository hosting platform. It allows developers to store, share, and synchronize project history with other developers.

- A local repository is stored on the developer's computer, while a remote repository is stored separately and can be accessed through a network.

Local Repository vs Remote Repository

Local Repository	Remote Repository
Stored on the developer's computer.	Stored on another system or repository hosting platform.
Can be used without continuously connecting to a network.	Usually accessed through a network connection.
Contains project files and GIT history.	Can contain a shared copy of project files and GIT history.
Used for local development and commits.	Used for sharing and synchronizing work.

Why Remote Repositories are Used

- To store a shared copy of a project.
- To back up project history outside the local computer.
- To share code with other developers.
- To synchronize work between developers.
- To support collaboration using branches and commits.
- To make a project accessible from multiple development environments.

Understanding Remote

A remote is a reference or name that represents a remote repository. The commonly used default remote name is origin.

```
origin  →  Remote Repository
```

The name origin is only a convenient name. A different remote name can also be used.

Connecting a Local Repository to a Remote Repository

Before pushing local commits to a remote repository, a local repository must be connected to the remote repository.

Basic Workflow

```
Local Repository
      ↓
Add Remote
      ↓
Remote Repository
      ↓
Push Changes
```

Adding a Remote Repository

Command

```
git remote add origin <repository-url>
```

Purpose of the Command

- Adds a remote repository and gives it the name origin.

Example

```
git remote add origin https://github.com/username/student-management.git
```

Here, origin is the remote name, and the URL identifies the remote repository.

Note

- The URL must point to the intended remote repository.
- The remote name origin is commonly used, but it is not mandatory.

Viewing Remote Repositories

Command

```
git remote -v
```

Purpose of the Command

- Displays the configured remote repository names and their URLs.

Example

```
git remote -v
```

```
origin  https://github.com/username/student-management.git
(fetch)
origin  https://github.com/username/student-management.git
(push)
```

Pushing Changes to a Remote Repository

Command

```
git push origin main
```

Purpose of the Command

- Sends local commits from the specified branch to the specified remote repository.

Example

```
git switch main
git push origin main
```

Here, origin is the remote name and main is the branch being pushed.

Pulling Changes from a Remote Repository

Command

```
git pull
```

Purpose of the Command

- Gets changes from the configured remote tracking branch and integrates them into the current local branch.

Example

```
git switch main
git pull
```

This is commonly used when the remote repository contains changes that are not yet present in the local branch.

Fetching Changes from a Remote Repository

Command

```
git fetch
```

Purpose of the Command

- Download new data and updates remote-tracking information without automatically integrating the changes into the current working branch.

Example

```
git fetch
git status
```

Fetch is useful when a developer wants to inspect remote changes before deciding how to integrate them.

Cloning a Remote Repository

Command

```
git clone <repository-url>
```

Purpose of the Command

- Creates a local copy of an existing remote repository, including its project files and GIT history.

Example

```
git clone https://github.com/username/student-management.git
```

A new local project directory is created from the remote repository.